# Software Engineering

## Reactive Systems I, ch 01~03

何明昕  HE Mingxin, Max

Send your email to c.max@yeah.net  with
a subject like:  *SE345-Andy: On What ...*

Download from c.program@yeah.net

/文件中心/网盘/SoftwareEngineering26S

# Slides for
# Design Methods for Reactive Systems:
# Yourdon, Statemate and the UML

**Roel Wieringa**
**Department of Computer Science**
**University of Twente,**
**the Netherlands**
**roelw@cs.utwente.nl**

**www.cs.utwente.nl/~roelw**

# List of Slides

# Chapter 1. Reactive Systems

- Partitioning of systems into information systems, control systems and telecommunication systems is becoming obsolete.

- More informative partitioning: Transformational versus reactive systems.

- Reactive systems respond to stimuli in order to bring about desirable effects in their environment.

- Reactive systems may do one or or of these things:

  - Manipulate complex data,
  - engage in complex behavior,
  - communicate with many other systems.

## Example 1

A training information system supports coordination of monthly introductory training courses for new employees of large company.

- *Nonterminating:* When switched on, it should respond to events such as queries, updates, arrival of downloads.
- *Interactive:* When switched on, it can engage in dialogs with its users and with other software (personnel information system).
- *State-dependent response:* Its response depends upon the data stored in it.
- *Environment-oriented response:* Responses defined in terms of courses, participants, teachers etc.
- *Parallel processing:* May interact with several users and programs simultaneously.

## Example 2

An electronic ticket system supports buying and using rail tickets by means of a smart card in combination with a PDA.

- *Nonterminating:* When switched on, it should respond to events such as buy, show, use.
- *Interactive:* When switched on, it can engage in dialogs with its users and with other software.
- *State-dependent response:* Its response depends upon the tickets and the data about railroads stored in it.
- *Environment-oriented response:* Responses defined in terms of railroad routes and segments.
- *Parallel processing:* It may consist of several pieces of software running concurrently on smart card and PDA.

## Example 3

A heating controller of heating tank in juice plant.

- *Nonterminating:* When switched on, it should respond to events such as start heating, too hot, too cold.
- *Interactive:* When switched on, it can engage in dialogs with its users and with devices.
- *Interrupt-driven:* It responds to time-outs and signals from operator and devices as and when they occur.
- *State-dependent response:* Its response depends upon the data that it stores, which is about the heating tank and its devices.
- *Environment-oriented response:* Responses are about heating tank and its devices.
- *Parallel processing:* Monitoring temperature, monitoring pressure, listening to commands from operator.
- *Real time:* Responses would be incorrect if too late.

## Definition of reactive system

A **reactive system** is a system that, when switched on, is able to create desired effects in its environment by enabling, enforcing or preventing events in the environment.

Has most of the following characteristics:

- nonterminating
- interactive
- interrupt-driven
- state-dependent
- environment-oriented
- parallel
- real-time

## Examples of reactive systems

- Information systems
- Workflow systems
  - Groupware
  - EDI systems
  - Web market places
  - Production control software
  - Embedded software

## Transformational systems

Contrast reactive systems with transformational systems, that compute output from an input and then terminate.

- terminating
- sometimes interactive
- not interrupt-driven
- output not state-dependent
- output defined in terms of input
- sequential
- usually not real-time

Compiler, assembler, loader, expert system, optimization algorithm, search algorithm, linear programming algorithm, etc.

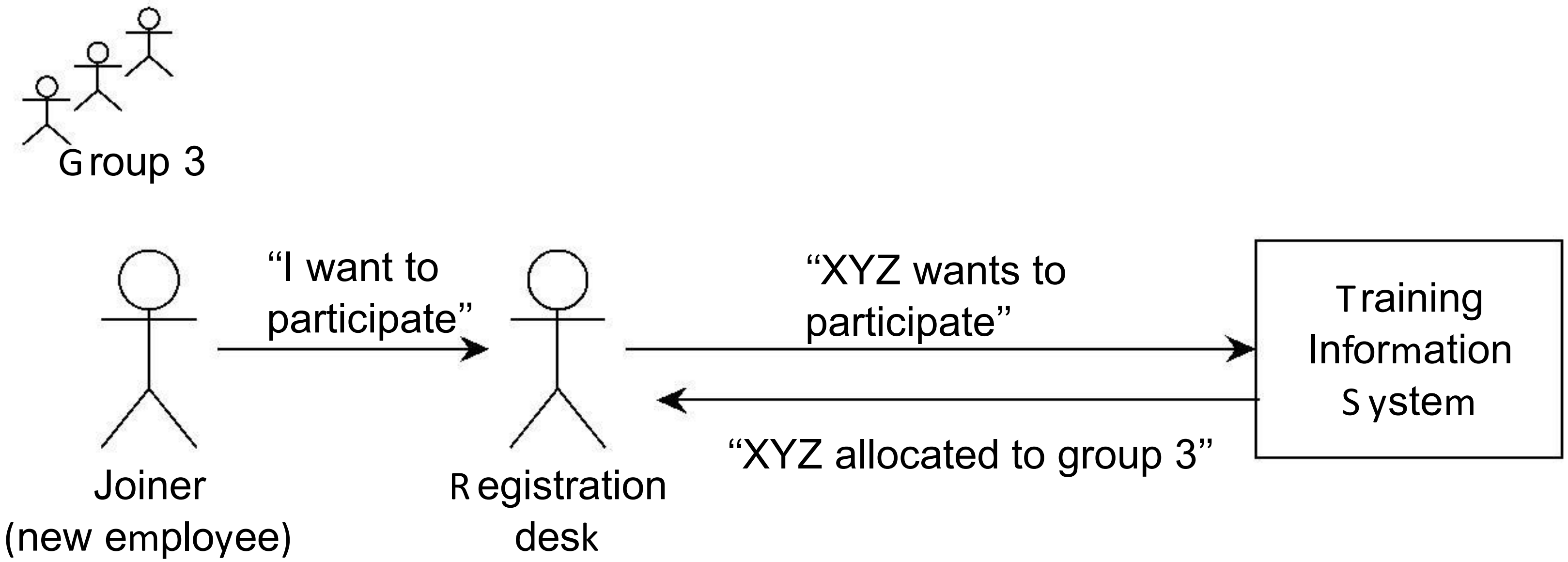
## Design approach to reactive systems

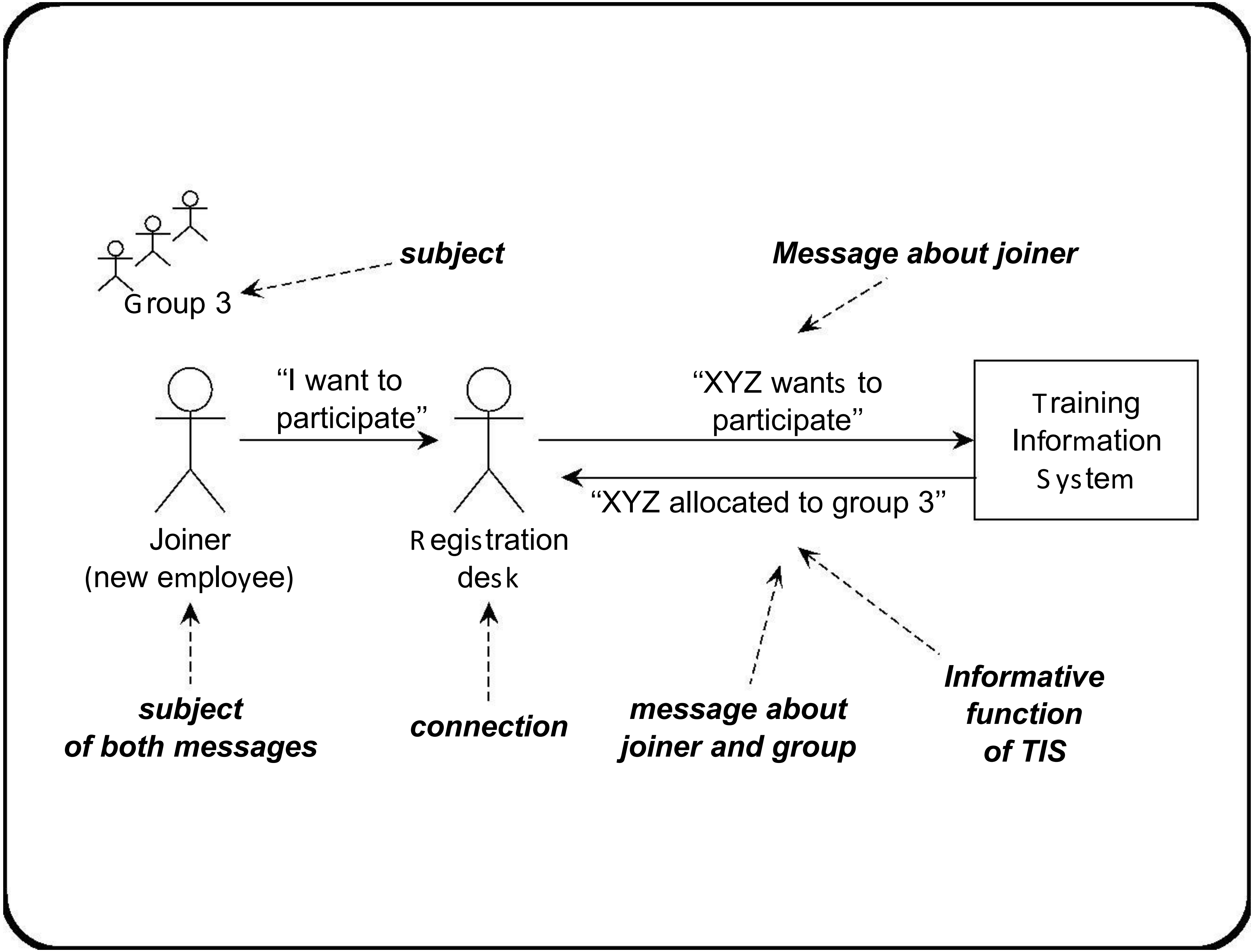
To design reactive systems, environment models are important:

- Possible entities and behavior in environment (chapter 2).
- Communication of system with environment (chapter 3)

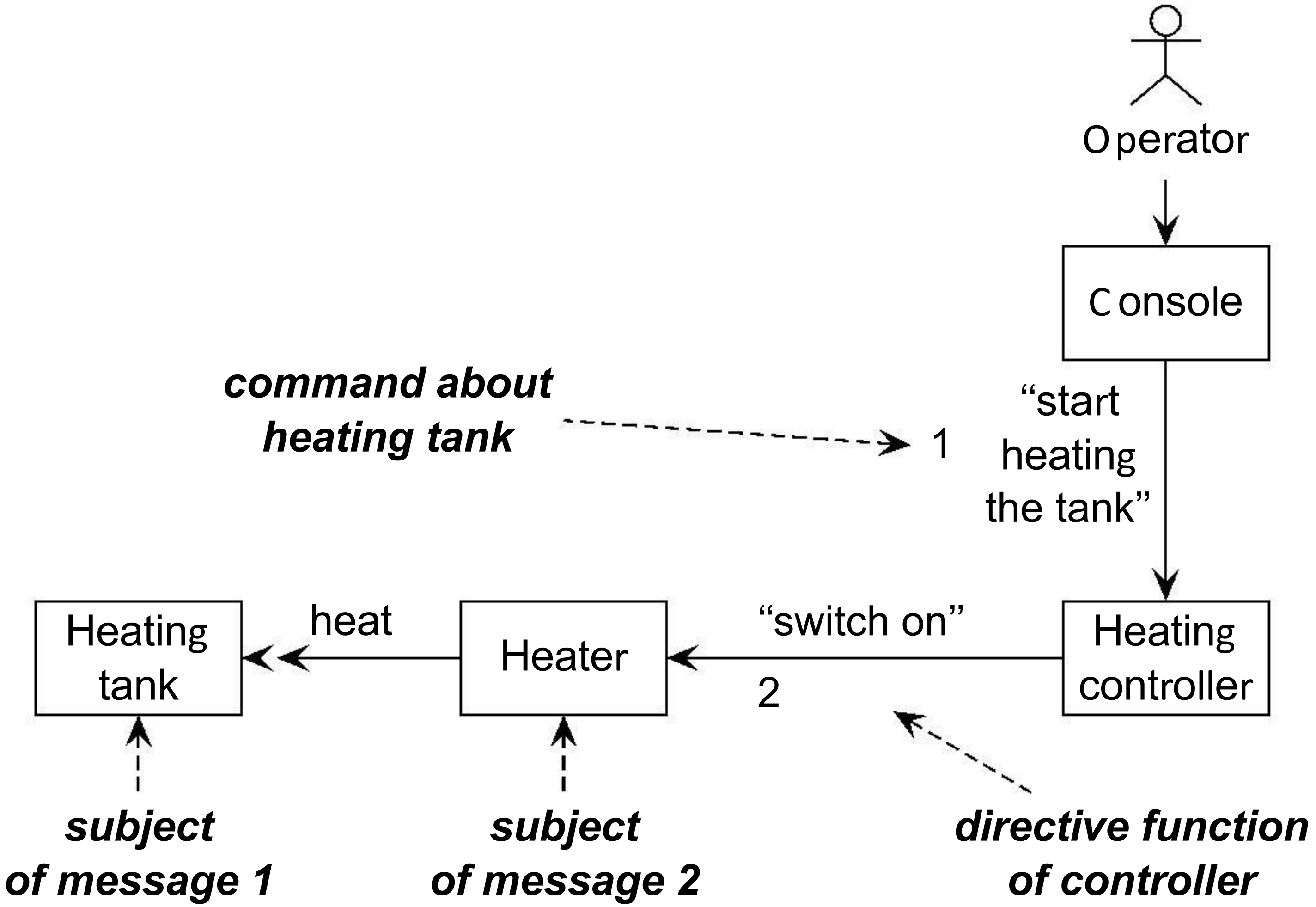
# Chapter 2. The Environment

## Example 1

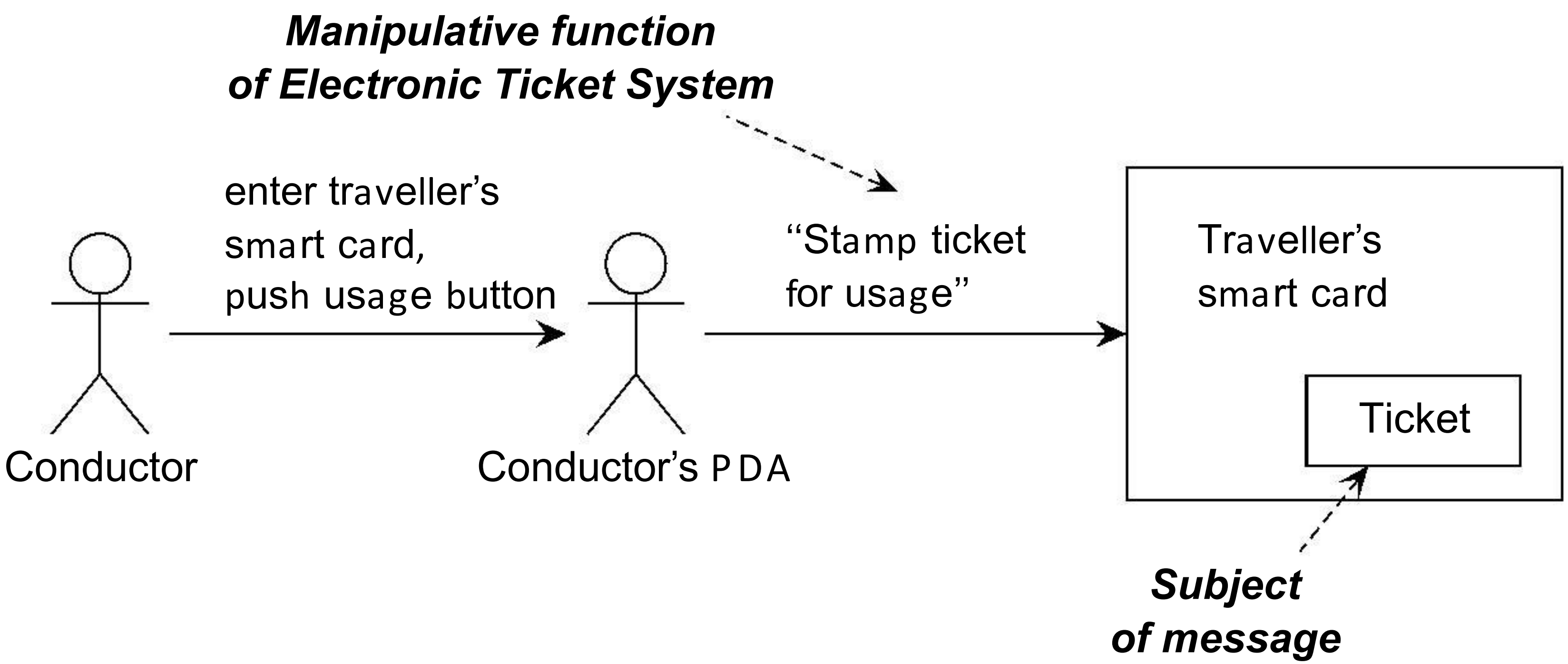



subject
Message about joiner
Group 3
"I want to participate"
"XYZ wants to participate"
Training Information System
"XYZ allocated to group 3"
Joiner (new employee)
Registration desk
subject of both messages
connection
message about joiner and group
Informative function of TIS

# Example 2


Operator
Console
command about heating tank
1
"start heating the tank"
Heating tank
heat
Heater
"switch on"
2
Heating controller
subject of message 1
subject of message 2
directive function of controller

## Example 3


Manipulative function
of Electronic Ticket System
enter traveller's
smart card,
push usage button
"Stamp ticket
for usage"
Traveller's
smart card
Ticket
Conductor
Conductor's PDA
Subject
of message

## Symbolic interactions

Reactive software systems communicate with their environment by means of linguistic messages.

- Not in object-oriented sense.
- Post-it notes exchanged with environment.
- Flow of symbols is more important than of flow of energy or matter.
- Physical connection is what makes symbol flow possible.
- Effect not determined by physical causality but by symbol flow.
- We abstract from the physical realization of these messages.

## Summary of examples

Messages entering and leaving a reactive software system are characterized by three aspects:

- **A subject**
  - What is the message about?

    About people, devices, conceptual entities, lexical entities.

- **A function**
  - What is the purpose of the message?

    To inform or direct the environment, to manipulate lexical items in the system.

- **A connection**
  - Through which path does the message travel from sender to receiver?

    Messages can get delayed, distorted or lost along the way.

## Definitions

- **Subject domain** of a reactive system: Set of all subjects of all its input and output messages.

- **Functions** of a reactive system: All services provided by these message exchanges.

- **Connection domain** of a reactive system: Channels through which messages flow to and from the system.

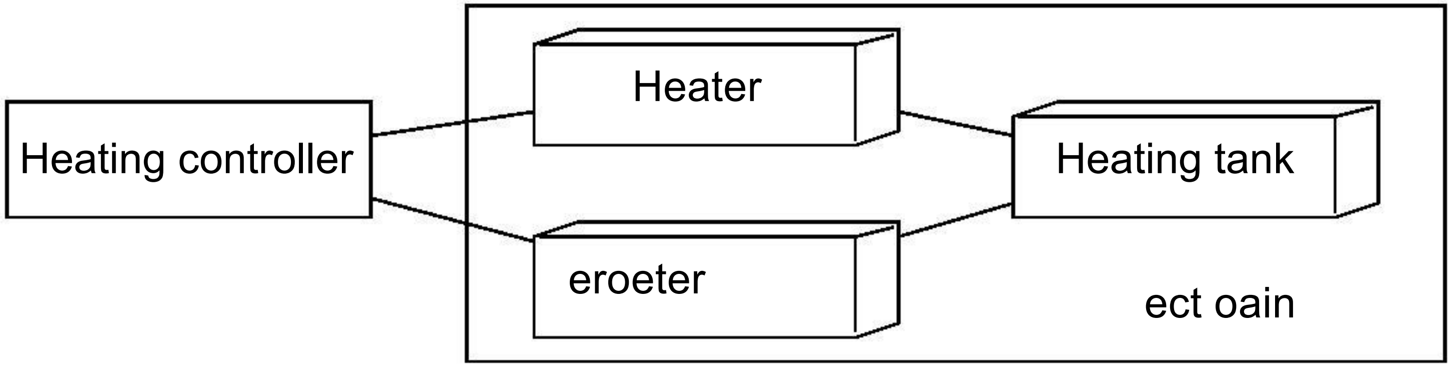
## Subject domain

The part of the world talked *about* by the messages that cross the system interface.

To find it, ask what the messages entering and leaving the system are *about.*

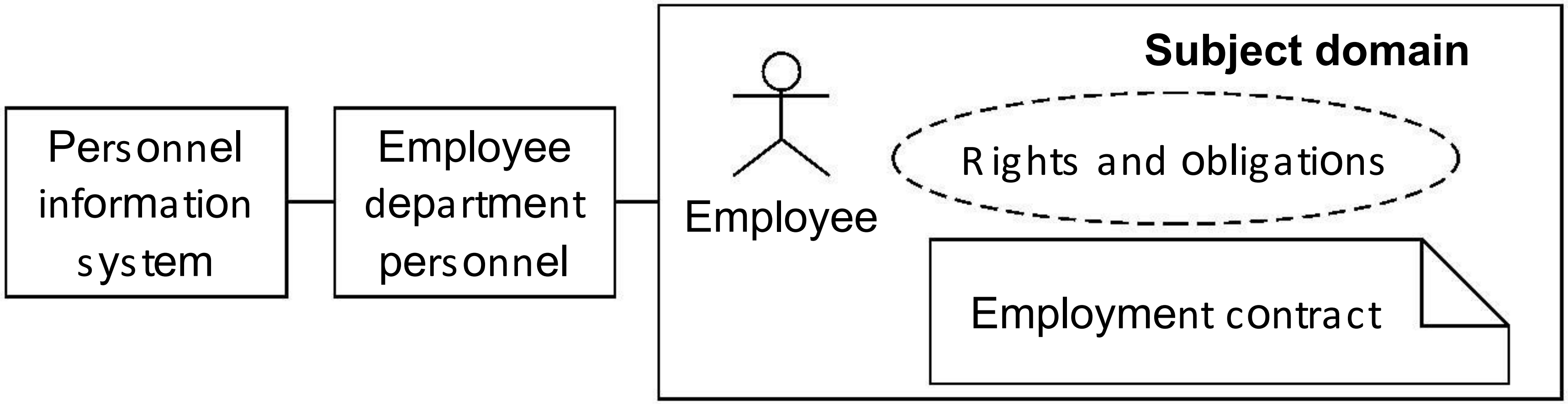
- **Physical entities.** Have a weight and a size. Make noise, generate heat.
  - People
  - Devices
- **Conceptual entities.** Invisible and weightless. E.g. bank accounts, obligations, permissions.
- **Lexical items.** Physical entities with a meaning. E.g. Tickets, contracts, receipts.

# Example of a physical subject domain



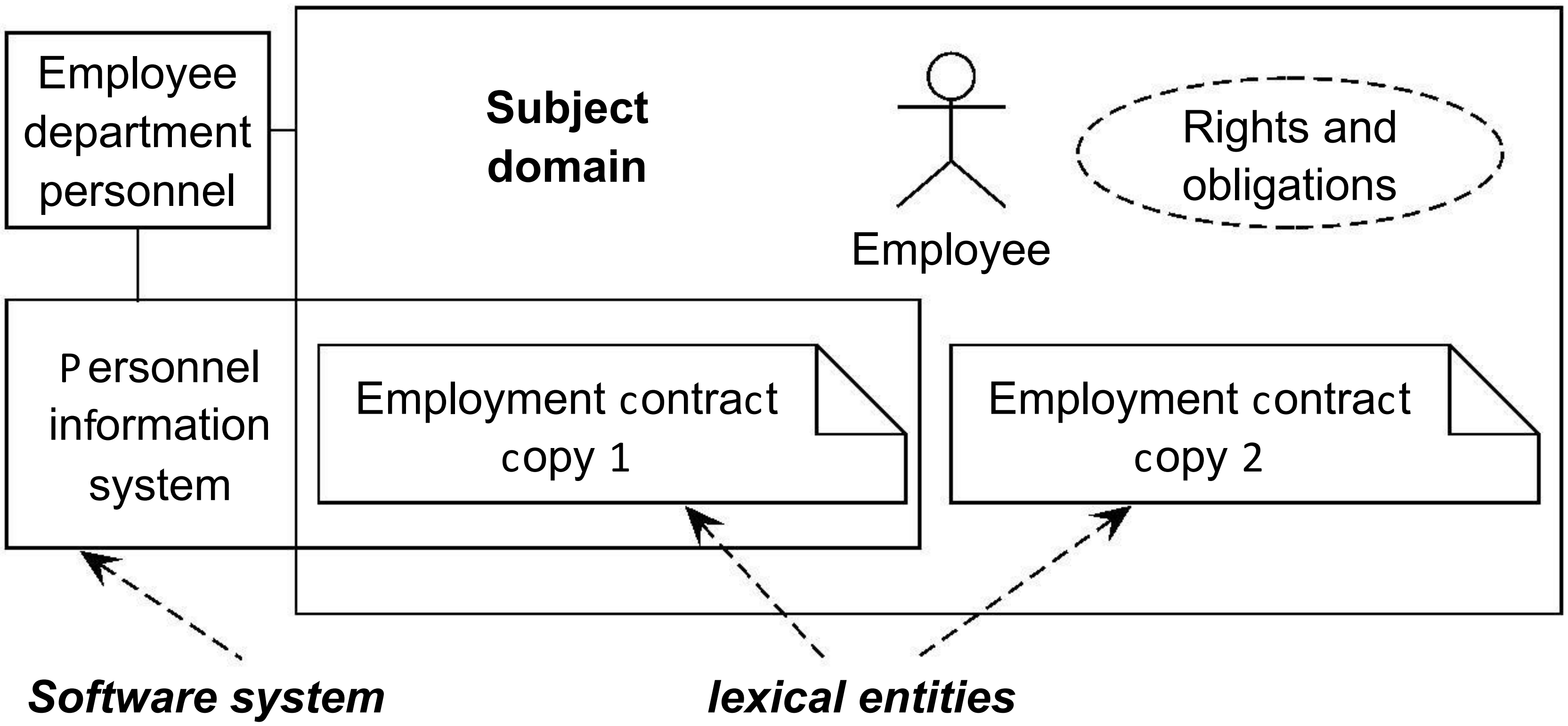

A physical subject domain always exists outside the software system.

## Example of conceptual, lexical, physical subject domain entities



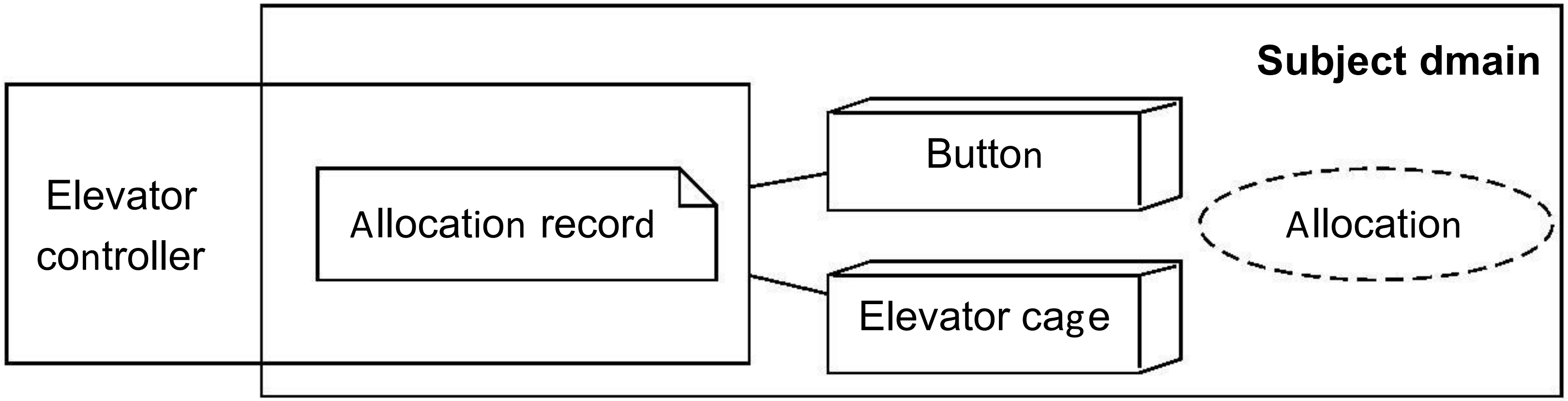

Conceptual entities always exist outside the software system. They exist because people agree to treat them as existing.

## Lexical entities can be copied




Lexical subject domain entities may exist in the software or in the environment of the software. They may be copies of each other.

# Physical and conceptual entities in a subject domain



## Functions of reactive systems

- **Registration.** System registers events in environment.
  - TIS registers participation in course.
  - Heating controller registers temperature.

- **Direction.** System influences events in environment.

  - Controller switches heater on.
  - Compact dynamic bus station controller: Tells driver at which platform to park.

- **Manipulation.** System manipulates virtual entities; this has a meaning in the social environment.
  - ETS stamps ticket.
  - TIS allocates joiner to group.

## Connection domain

- Events of interest usually do not occur *exactly* at the interface of the system.

- Actions caused by the system usually do not occur *exactly* at the interface of the system.

Communication channel needed. If represented explicitly it is called a **connection domain.** Is possible source of

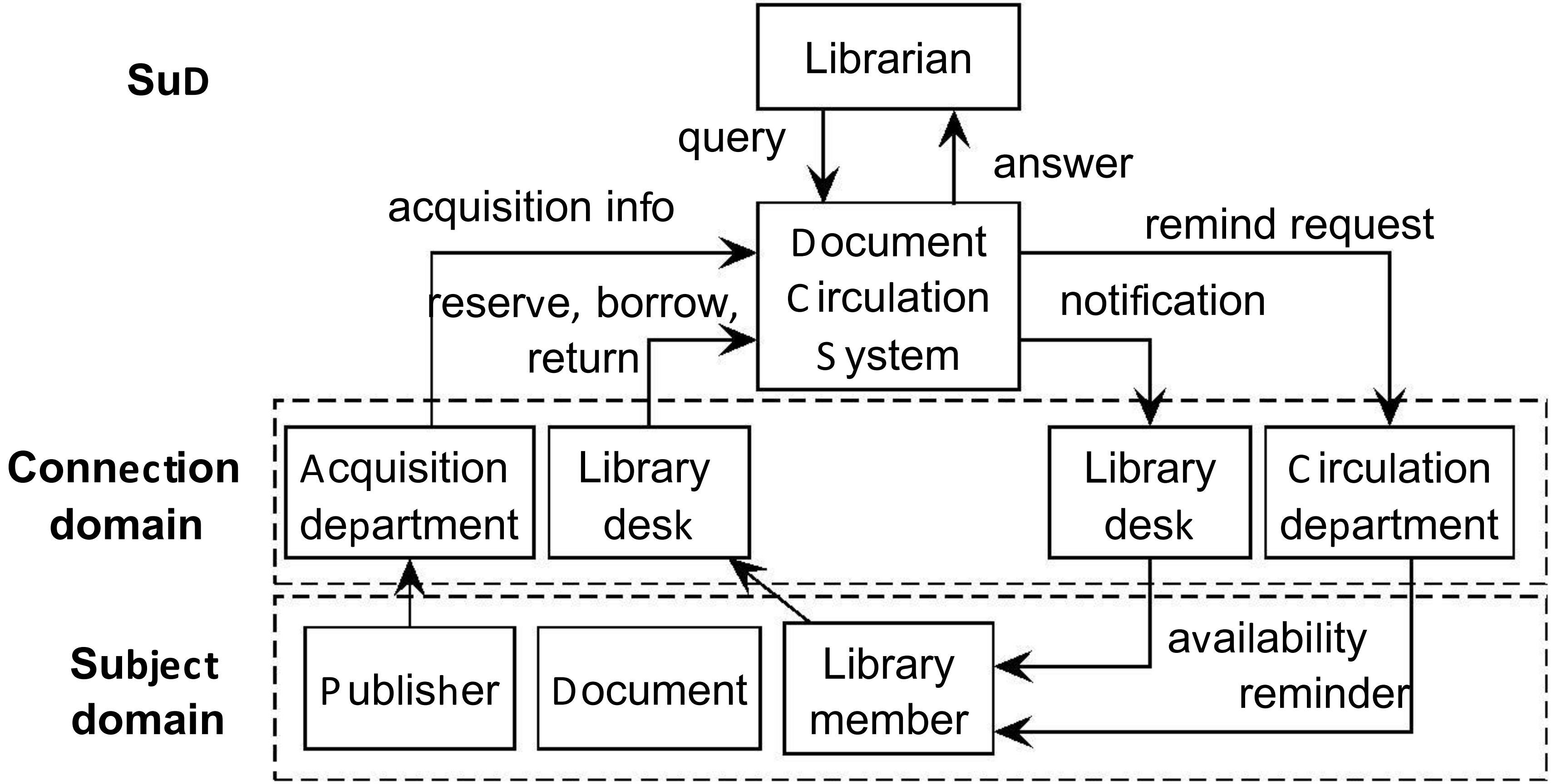
- delay,

- loss, and

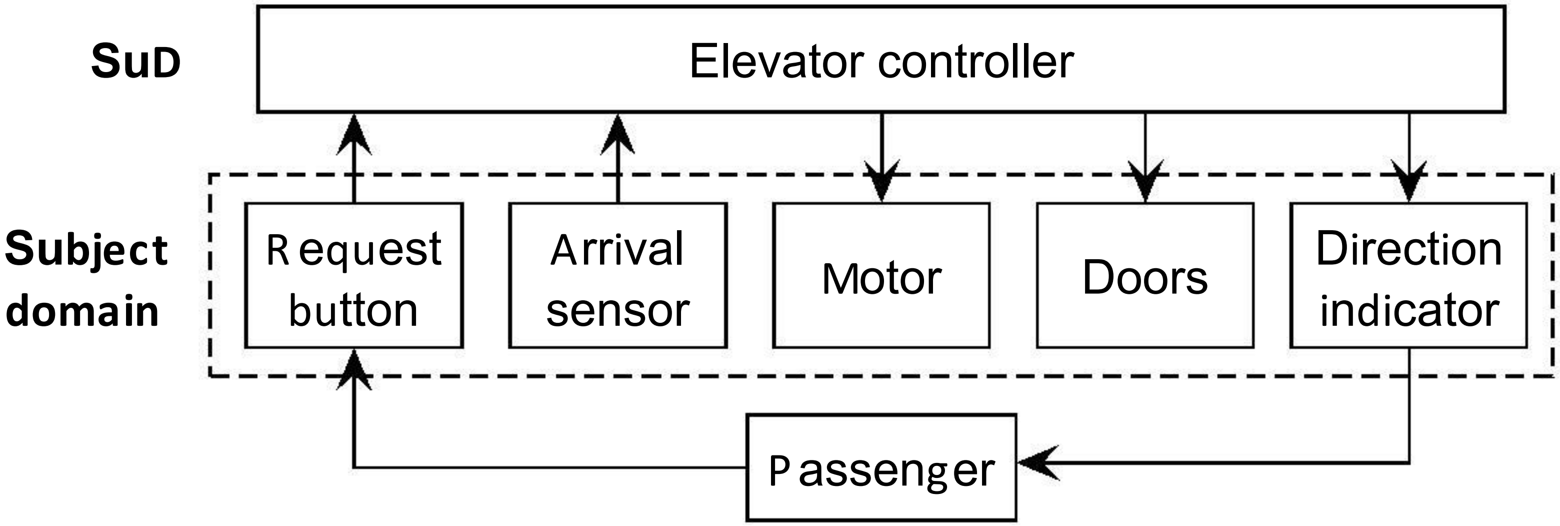
- distortion

of messages to/from the system.

# Subject domain and connection domain



## System directly connected to subject domain



## Main Points

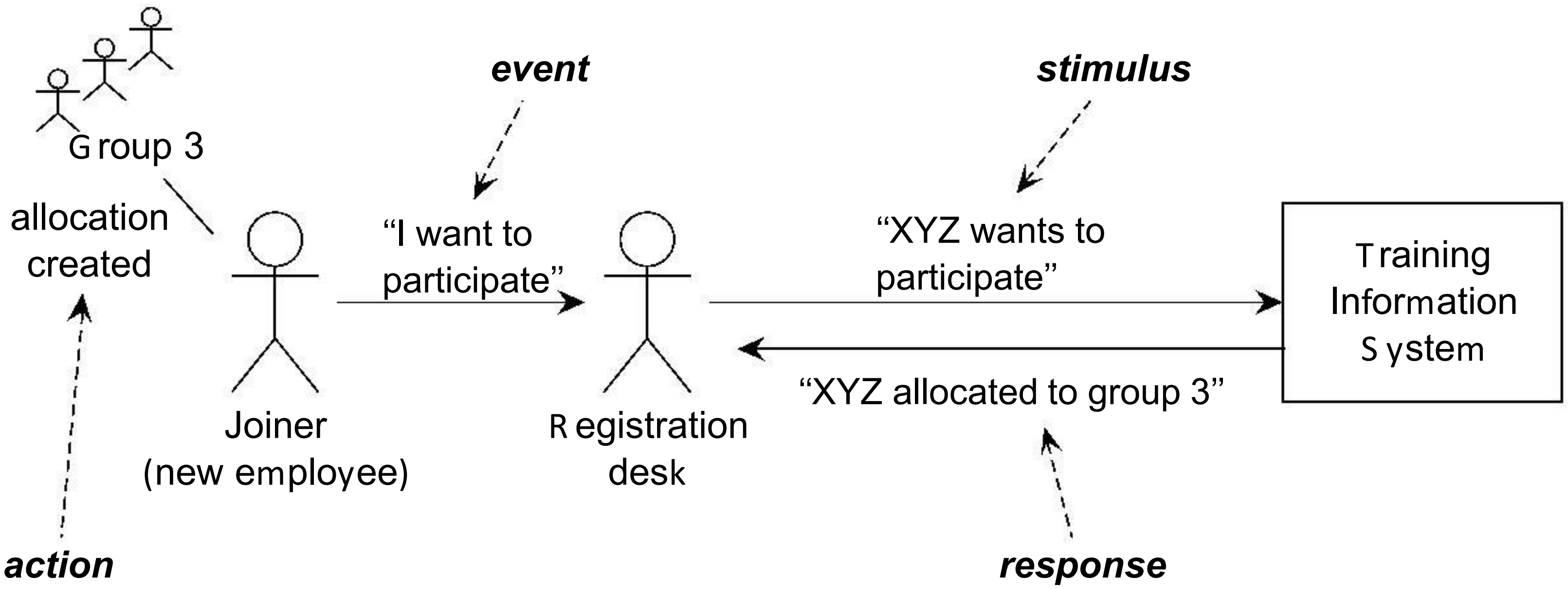
Let SuD be a Software System under Design.

- Message that cross the interface of the SuD are *about* the subject domain.
- Subject domain entities exist *outside* the SuD; except lexical items, that may also exist inside the SuD.
- Messages that cross the interface have three kinds of functions: *Informative, Manipulative, Directive.*
- Messages travel to and from the SuD through a *connection domain.*

# Chapter 3. Stimulus-Response Behavior

- A reactive system receives messages from sources in its environment and it sends messages to destinations in its environment.
- At the sources, events occur.
- At the destinations, actions occur.
- We now look at the chain of cause and effect from event to system stimulus, and from system response to action.
- We will see that to interpret stimuli and to motivate responses, we need to make assumptions about the environment.

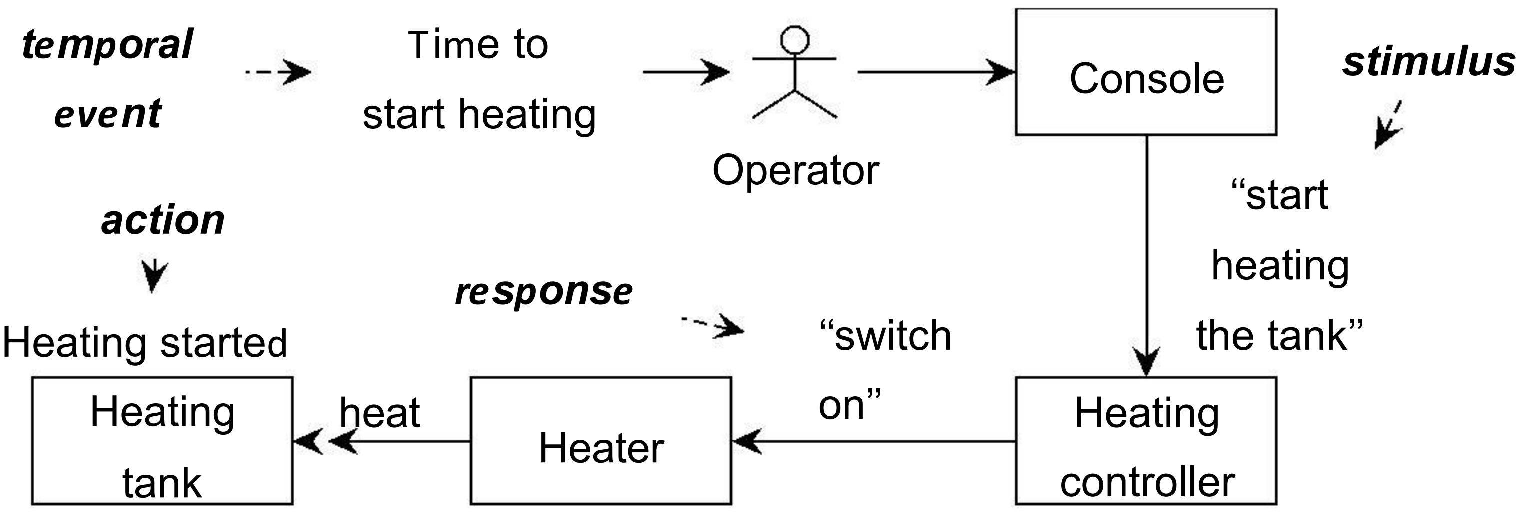
## Example 1




Assumptions:

- Registration desk transmits message faithfully.
- The employee is indeed a joiner.

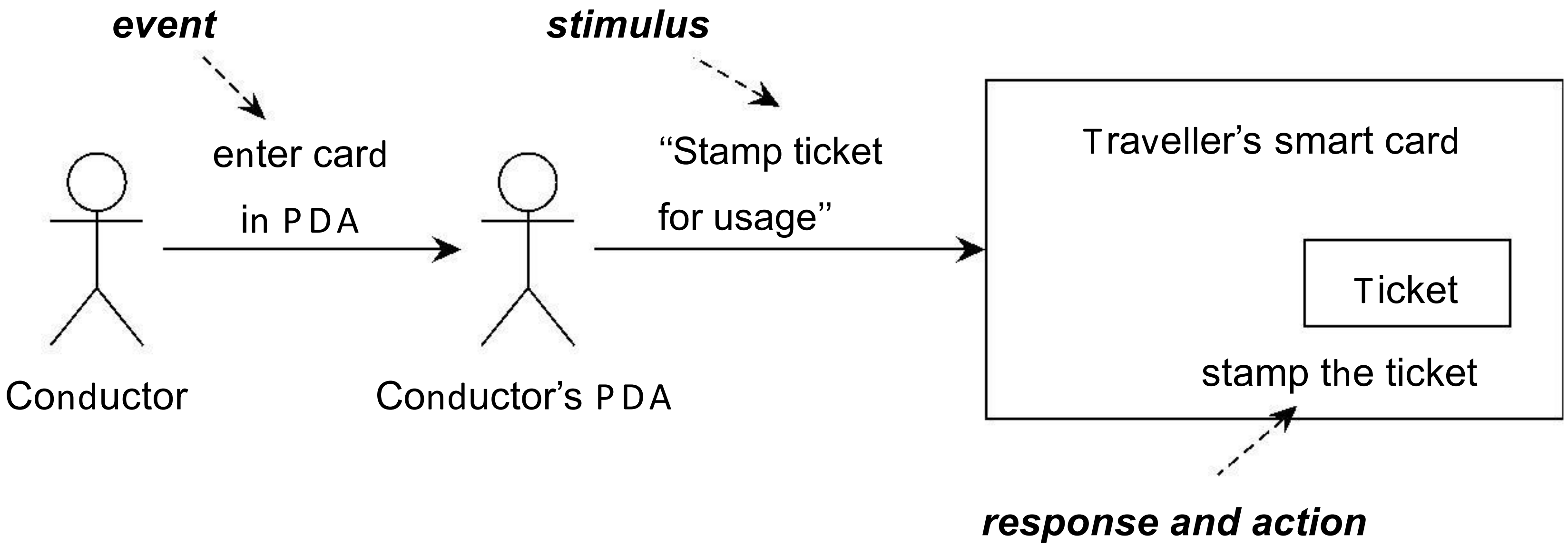
# Example 2




Assumptions:

- The message is transmitted faithfully by the console.
- The translates "switch on" into heat production.
- The heater is connected to the heating tank.

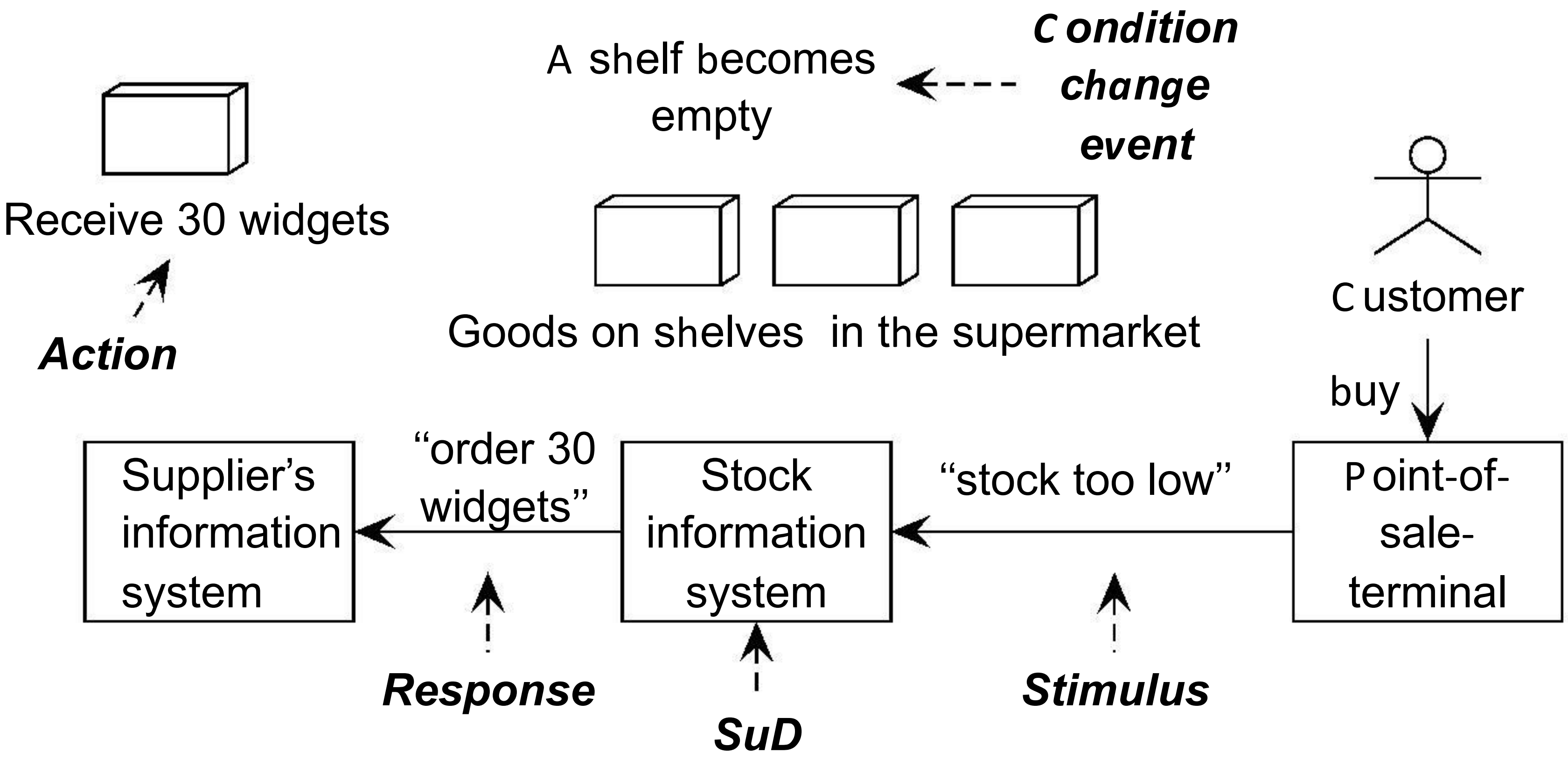
# Example 3




Assumptions:

- The PDA is indeed the conductor's PDA.
- The conductor requests to stamp the ticket while the ticket owner is traveling on a segment for which the ticket is valid.

## Example 4




SuD discovers the change event by testing the condition stock $<$ 4.

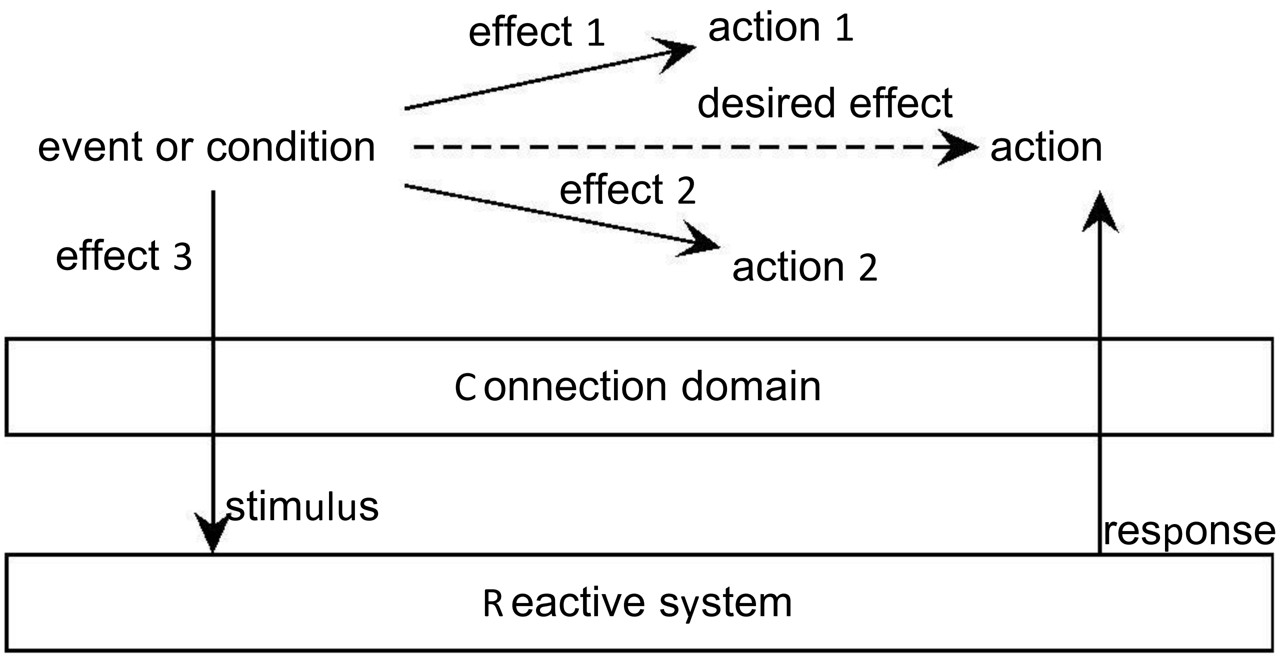
Assumptions:

- Data in SIS accurately represent state of the shelves.

## Summary of stimulus-response behavior

- Some *named external event, condition change event* or *temporal event* occurs in the environment.
- Through some some communication channel, this causes an SuD *stimulus.*
  - May be modeled explicitly as connection domain.
- The SuD *responds.*
- Through some communication channel, this causes an *action* in the environment.
  - For virtual entities, response and action coincide.

# Structure of stimulus-response behavior



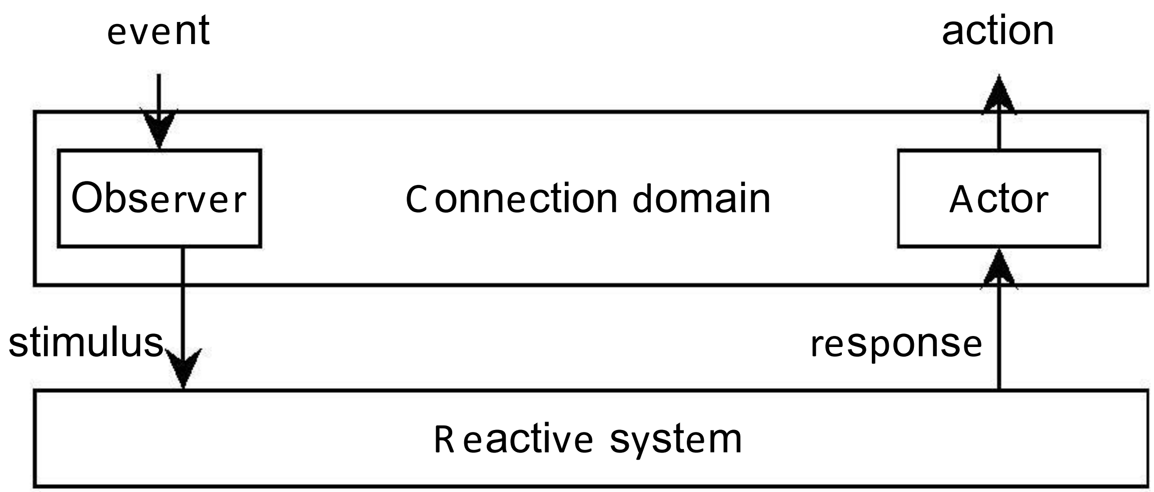
## The role of assumptions

These are statements about the environment ...

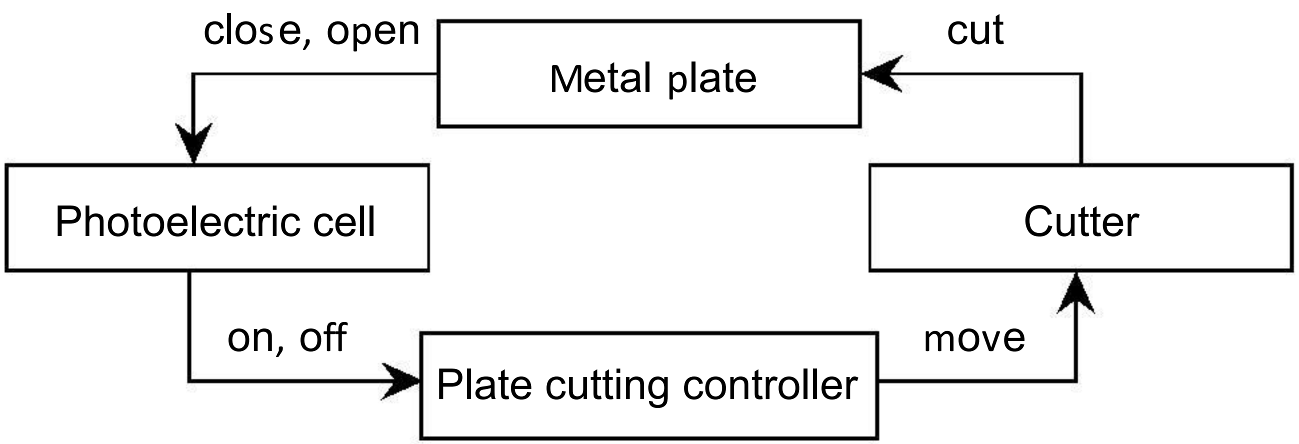
- that must be true for the (stimulus, response) pair to be desirable;
- but the SuD cannot guarantee them to be true.
- If assumption is false, (stimulus, response) pair may still be desirable, or indifferent, or even undesirable!

Typical assumptions concern laws of nature and properties of devices (e.g. observers and actors) and people (e.g. users and operators).

## Observers and actors are in the connection domain



# Assumptions about observers




- Events of interest: begin of plate arrives, end of plate arrives
- Available stimuli: on, off.

We need the following assumptions:

- The photo-electric cell is functioning properly.
- The cell is stimulated only by the arrival of the begin or end of a metal plate.

## Event recognition

System must respond to the event

- a point arrives where the sheet must be cut.

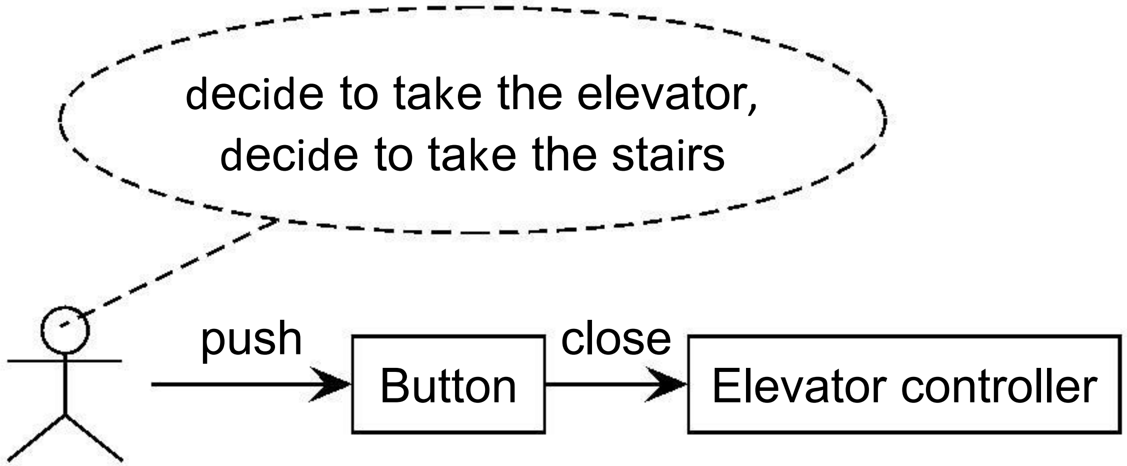
The system must infer the occurrence of this event. **Event recognition:**

- Record the point in time at which plate arrives.
- Wait for time at which desired point is under the cutter.

Additional assumption needed:

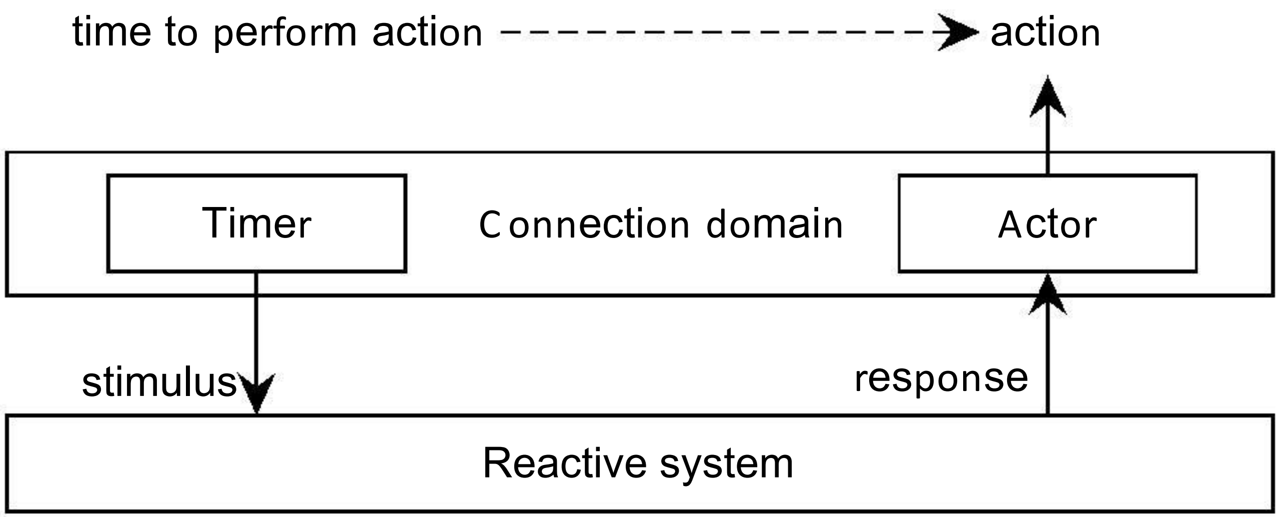
- Speed of the plate is $n$ meters / second.

# Observability of events

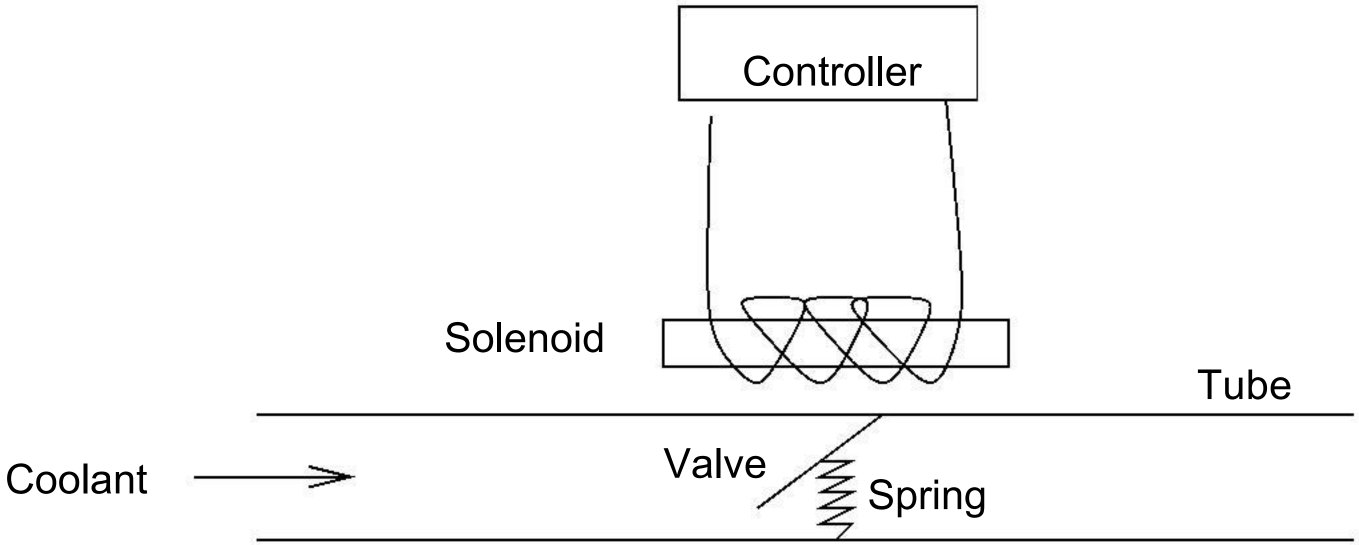



What is the subject domain of the elevator controller?

## Observing temporal events



# Realizability of actions



## Main points

- Reactive software system is connected to interesting events and actions by a communication channel.

- Stimulus is event observation.

- Response is assumed to cause desired action.

- If assumptions about environment are true, (stimulus, response) pairs are desirable; otherwise, they may be undesirable or in different.

- Assumptions cannot be guaranteed by the system.